

Predicting Final Chess Game Outcomes from Move-Level Board States

Ivan Chabanov – Data Engineer
Alexander Mikhailov – Data Scientist
Danil Eramasov – Technical Writer
Bulat Fakhrutdinov – ML Specialist

Big Data Course – IU
2026

Contents

1	Introduction	4
2	Business Objectives	4
3	Data Description	4
3.1	Feature Categories	5
4	Data Characteristics	5
5	Architecture of the Data Pipeline	6
5.1	Overview	6
5.2	Pipeline Flow Diagram	6
6	Data Preparation	6
6.1	ER Diagram	6
6.2	Loading Data to PostgreSQL	6
7	Stage I – Data Ingestion	6
7.1	Objectives	6
7.2	PostgreSQL Setup	7
7.3	Sqoop Import to HDFS	7
7.4	Deliverables	7
8	Stage II – Data Storage and Exploratory Data Analysis	7
8.1	Creating Hive Tables	7
8.2	Exploratory Data Analysis	7
8.2.1	Insight 1: White Win Percentage by Opening (ECO Code)	8
8.2.2	Insight 2: Effect of Rating Gap on Game Outcome	8
8.2.3	Insight 3: Average Game Length by Time Control	8
8.2.4	Insight 4: Distribution of Checkmate Plies by Game Phase	8
8.2.5	Insight 5: Outcome Trends Over Time (by Archive Month)	9
8.2.6	Insight 6: Material Advantage and White Win Probability	9
9	Stage III – Predictive Data Analysis	9
9.1	Objectives	9
9.2	Feature Extraction and Preprocessing	9
9.3	Train-Test Split	9
9.4	Models	10
9.4.1	Model 1: Random Forest	10
9.4.2	Model 2: Logistic regression	10
9.4.3	Model 3: Naive Bayes	10
9.5	k-Fold Cross Validation	10
9.6	Evaluation Metrics	10
9.7	Best Model Selection	11
9.8	Prediction Example	11
10	Stage IV – Data Presentation (Dashboard)	11
10.1	Dashboard Description	11

11 Findings and Conclusion	12
11.1 Summary	12
11.2 Reflections on Own Work	12
11.3 Challenges and Difficulties	12
11.4 Recommendations	13
12 Contribution Table	13

1 Introduction

The rapid growth of digital platforms has generated massive volumes of structured and semi-structured data requiring scalable storage and processing solutions. Online chess platforms such as Chess.com produce millions of games monthly, each consisting of detailed move-by-move records enriched with contextual and player-related metadata.

This project aims to design and implement an end-to-end big data pipeline capable of ingesting, storing, analyzing, and modeling large-scale chess game data. The pipeline integrates distributed storage (HDFS), relational databases (PostgreSQL), Hive-based warehousing, Spark-based distributed analytics, and interactive visualization via Apache Superset.

The core analytical objective is to predict the final outcome of a chess game (White win, Black win, or Draw) using board state features and contextual information available at a given ply. This represents a multiclass classification problem executed in a distributed Spark environment.

2 Business Objectives

Although chess is a strategic board game, predictive modeling of game outcomes has significant analytical and practical implications:

- Improving real-time win probability estimation.
- Enhancing chess analytics platforms.
- Supporting training and educational tools for players.
- Identifying decisive positional patterns.
- Studying the influence of rating gaps and time controls on game outcomes.

The primary objective of this project is:

To build a scalable distributed system that predicts the final outcome of a chess game using intermediate board-state features and contextual metadata.

Secondary objectives include understanding how positional advantages, time pressure, and rating differences influence the final result of a game.

3 Data Description

The dataset consists of chess game records collected from Chess.com archives. The raw sequential PGN (Portable Game Notation) archives were transformed into a structured relational format suitable for distributed storage and analytics.

Each observation corresponds either to:

- A **game-level record** containing metadata (players, ratings, time control, result).
- A **ply-level record** representing the board state at a specific move.

The dataset contains more than **660,000 plies** across thousands of games.

3.1 Feature Categories

The dataset includes the following types of features:

- Player ratings (`white_rating`, `black_rating`)
- Rating gap
- Time control class (blitz, rapid, classical, etc.)
- ECO (Encyclopedia of Chess Openings) code
- Archive month (datetime feature)
- Material balance indicators
- Pawn structure features
- King safety indicators
- Move number and game phase (opening, middlegame, endgame)
- Termination type (checkmate, resignation, timeout, agreement)
- Final result class (**target variable**: White win, Black win, Draw)

4 Data Characteristics

Property	Value
Number of plies	> 660,000
Dataset size	> 500 MB
Number of features	> 100
Target variable	<code>final_result_class</code>
Task type	Multiclass Classification
Datetime feature	<code>archive_month</code>
Geospatial feature	No
Storage format	CSV → PostgreSQL → HDFS → Hive

Table 1: Dataset characteristics

The dataset satisfies all project requirements: it exceeds 500,000 records, contains more than six explanatory variables, includes a datetime feature, and supports large-scale distributed batch processing.

5 Architecture of the Data Pipeline

The project implements a full end-to-end big data pipeline consisting of four major stages.

5.1 Overview

The data flows through the following components:

1. **Data Source:** Chess.com PGN archives
2. **Relational Database:** PostgreSQL
3. **Distributed Storage:** HDFS (Hadoop Distributed File System)
4. **Data Warehouse:** Apache Hive (columnar storage with compression)
5. **Processing Engine:** Apache Spark on YARN
6. **Visualization:** Apache Superset

5.2 Pipeline Flow Diagram

Raw PGN --> PostgreSQL --> Sqoop --> HDFS --> Hive --> Spark MLlib
--> Superset

6 Data Preparation

6.1 ER Diagram

The relational model consists of the one primary table:

- `chess_moves` – stores moves across all games

6.2 Loading Data to PostgreSQL

The schema for the table was created using an autogenerated alembic migration to ensure declared types matched created ones.

Data was loaded into PostgreSQL using the `scripts/load_data.py` script.

Ingestion logs and schema creation results are stored in:

- `output/postgres_results.txt`

7 Stage I – Data Ingestion

7.1 Objectives

- Load structured chess data into a distributed relational database.
- Use Sqoop to transfer data from PostgreSQL to HDFS.
- Prepare data for Hive table creation.

7.2 PostgreSQL Setup

Tables were created with appropriate schema, constraints, and indexes to support efficient querying.

7.3 Sqoop Import to HDFS

We first created a table of `chess_moves_raw` to import raw data from Postgres with the intention of optimizing it later and stored in PARQUET format. This format was chosen as its column-level format enables faster query performance, especially for our analytical and ML workflow. Besides, it didn't require maintaining an additional schema file.

Apache Sqoop was used to import data from PostgreSQL into HDFS.

The data is now stored in HDFS in compressed PARQUET format, ready for Hive integration.

7.4 Deliverables

- All tables built and loaded in PostgreSQL
- Data successfully imported to HDFS via Sqoop
- The table was optimized

8 Stage II – Data Storage and Exploratory Data Analysis

8.1 Creating Hive Tables

External Hive tables were created on top of the HDFS data imported via Sqoop.

The main `chess_moves` table was partitioned by `archive_month`, and divided into 8 buckets by `game_uuid` for optimizations. Partitioning by archived date made sense as many of the data insights could be co-related to the time a game was played. Logically, this division makes sense for a distributed storage, as older entries can be compressed to free storage.

The `chess_moves` table was populated from `chess_moves_raw` via an `INSERT OVERWRITE`.

This ensures efficient storage and fast analytical queries using columnar compression.

Logs of table creation are stored in `output/hive_results.txt`.

8.2 Exploratory Data Analysis

Eight analytical insights were created using HiveQL queries executed on the Tez engine. Each query produces a CSV output and has its own Superset Chart m

8.2.1 Insight 1: White Win Percentage by Opening (ECO Code)

Query: sql/q2.hql **Output:** output/q2.csv **Chart:** Horizontal bar chart (top 15 openings)

Description:

This query shows the statistics of the games after the round openings.

The analysis helps to determine which openings are more productive and which are less productive at the beginning of the game. The percentage of wins for white, black, draws and the total number of games is indicated.

Business Value:

Players and coaches can prioritize studying high-impact openings. Platforms can highlight popular and effective openings in training modules.

8.2.2 Insight 2: Effect of Rating Gap on Game Outcome

Query: sql/q3.hql **Output:** output/q3.csv **Chart:** Line chart (2 charts, 3 series: white win %, draw %, black win %)

Description:

This 2 queries show how much the difference in rating affects the outcome of the games. GAP is calculated based on the rating difference (white - black). If the GAP is negative, then black is stronger than white, if the GAP is positive, then on the contrary.

Business Value:

This supports fair matchmaking and accurate Elo rating adjustments.

8.2.3 Insight 3: Average Game Length by Time Control

Query: sql/q4.hql **Output:** output/q4.csv **Chart:** Bar chart

Description:

This diagram shows the statistics of batches of different time modes.

Business Value:

The analysis helps to determine how different game modes were played. Bullet (1 min), blitz (5 min), rapid (10 min), daily (unlimited). The average number of moves, the minimum and maximum number.

8.2.4 Insight 4: Distribution of Checkmate Plies by Game Phase

Query: sql/q5.hql **Output:** output/q5.csv **Chart:** Pie chart

Description:

This chart shows the number of piles of won game in a given period of the game.

Business Value:

The analysis helps to determine on what ply-index on average each game phase ends. We see, that in endgame game usually ends on 100-th ply, 54-th ply in middlegame, 14-th ply in opening.

8.2.5 Insight 5: Outcome Trends Over Time (by Archive Month)

Query: `sql/q6.hql` **Output:** `output/q6.csv` **Chart:** Line chart

Description:

This insight tracks outcome distribution over time using the `archive_month` datetime feature.

Business Value:

Confirms dataset stability and suitability for long-term modeling.

8.2.6 Insight 6: Material Advantage and White Win Probability

Query: `sql/q8.hql` **Output:** `output/q8.csv` **Chart:** Line chart

Description:

This query shows the chances of winning a game for white if they had an advantage by the middle of the game or if they were losing.

Business Value:

The analysis helps to determine how much the advantage of the pieces can affect the outcome of the game. Usually, the greater the advantage, the higher the chance of winning.

9 Stage III – Predictive Data Analysis

9.1 Objectives

- Build distributed machine learning models using Spark MLlib.
- Predict game outcomes using ply-level features.
- Optimize hyperparameters via grid search.
- Evaluate models using accuracy and F1-score.

9.2 Feature Extraction and Preprocessing

Features were extracted from the `plies` table and transformed using Spark DataFrame API:

- Categorical encoding for `time_class`, `eco_code`, `phase`
- Numerical scaling for `material_balance`, `king_safety`
- Feature vectorization using `VectorAssembler`

9.3 Train-Test Split

The dataset was split into:

- 70% training
- 30% testing

9.4 Models

Three types of classifiers were trained:

9.4.1 Model 1: Random Forest

Hyperparameters tuned:

- `numTrees`: [20, 40, 60]
- `maxDepth`: [3, 6, 9]
- `minInstancesPerNode`: [5, 10, 20]

Total combinations: 27

9.4.2 Model 2: Logistic regression

Hyperparameters tuned:

- `regParam`: [0.01, 0.1, 1.0]
- `elasticNetParam`: [0.0, 0.5, 1]
- `tol`: [1e-4, 1e-3, 1e-2]

9.4.3 Model 3: Naive Bayes

Hyperparameters tuned:

- `smoothing`: [0.5, 1.0, 2]
- `modelType`: [multinomial, bernoulli, gaussian]
- `threshold`: [0, 0.5, 1]

9.5 k-Fold Cross Validation

k-fold cross-validation was performed with $k = 3$.

9.6 Evaluation Metrics

Since this is a multiclass classification task, the following metrics were computed:

- **Accuracy**
- **F1-score (weighted)**

9.7 Best Model Selection

The best model of each type was selected based on cross-validation performance. Final evaluation was performed on the test set.

Results:

Model	Accuracy	F1-Score
Random Forest	0,70	0,68
Logistic regression	0,68	0,66
Naive Bayes	0,68	0,65

Table 2: Model performance comparison

9.8 Prediction Example

A sample ply was selected, and the best model was used to predict the final outcome:

Input features:

- Material balance: +2
- King safety: 4
- Phase: middlegame
- Time class: blitz

Predicted outcome: White win

10 Stage IV – Data Presentation (Dashboard)

10.1 Dashboard Description

A unified Apache Superset dashboard was created to present:

- Dataset characteristics (number of games, plies, features)
- All 8 EDA insights with charts
- Model performance metrics
- Example prediction

Each chart includes:

- A descriptive title
- Axis labels
- Data storytelling interpretation

The dashboard is visually clean, uses consistent color schemes, and supports interactive filtering.

11 Findings and Conclusion

11.1 Summary

This project successfully implemented an end-to-end big data pipeline for analyzing and predicting chess game outcomes. The pipeline integrated:

- Distributed relational storage (PostgreSQL)
- Scalable file storage (HDFS)
- Columnar data warehousing (Hive with Parquet + Snappy)
- Distributed analytics (Spark SQL and MLlib on YARN)
- Interactive visualization (Apache Superset)

The exploratory data analysis revealed valuable insights into how time controls, openings, rating gaps, and material balance influence game outcomes.

Three machine learning models were trained and optimized, demonstrating that distributed ML on Spark can effectively handle large-scale multiclass classification tasks.

11.2 Reflections on Own Work

The team successfully collaborated across roles:

- Ivan Chabanov led the data engineering pipeline (PostgreSQL, Sqoop, HDFS).
- Alexander Mikhailov performed exploratory analysis and designed queries.
- Danil Eramasov documented the process and created storytelling narratives.
- Bulat Fakhрутdinov implemented and tuned the ML models.

11.3 Challenges and Difficulties

- **Data transformation:** Converting PGN archives into a relational schema required custom parsing logic.
- **Sqoop configuration:** Tuning parallelism and compression settings for optimal performance.
- **Hive query optimization:** Some queries required partitioning and bucketing for efficiency.
- **Hyperparameter tuning:** Grid search on Spark required significant cluster resources.

11.4 Recommendations

- Future work could incorporate additional features such as piece activity and pawn chains.
- Time-series forecasting models could predict rating trends.
- Deep learning models (e.g., neural networks) may improve classification performance.
- Real-time streaming pipelines could support live game prediction.

12 Contribution Table

Task	Ivan	Alex	Danil	Bulat	Deliverables	Avg Hours
Data extraction	85%	5%	5%	5%	CSV files	3
PostgreSQL setup	5%	85%	5%	5%	postgres_results.txt	4
Sqoop import	5%	85%	5%	5%	HDFS datasets	3
Hive tables	5%	85%	5%	5%	hive_results.txt	1
EDA queries	85%	5%	5%	5%	q1.csv-q8.csv	3
Superset charts	5%	30%	60%	5%	Charts	2
Feature engineering	5%	5%	5%	85%	feature vectors	6
Model training	5%	5%	5%	85%	trained models	10
Hyperparameter tuning	5%	5%	5%	85%	best models	8
Dashboard creation	5%	60%	30%	5%	dashboard	5
Report writing	15%	20%	50%	15%	report.pdf	10
Presentation	20%	10%	60%	10%	presentation2.pdf	4

Table 3: Team contribution table